Teacher's Assessment Activity

DATA STRUCTURE AND ALGORITHMS

24EE05TH0505-2

5th Semester B.Tech. Session-2026-27

TA Report

RAMDEOBABA UNIVERSITY, NAGPUR
Formerly Shri Ramdeobaba College of Engineering & Management (RCOEM) Est. 1984
LEARN | INNOVATE | ACCOMPLISH

Title : E-Commerce Product Search

Project Student : Shravan Mungelwar (Roll no : 43 / Section B)

Course Coordinator

Prof. Sameer Tembhurney

Assistant Professor

Computer Science Engineering Department,

Ramdeobaba University, Nagpur

Contents

1. Introduction

2. Problem Statement and Objectives

3. System Requirements and Technologies Used

4. Architecture Diagram

5. Implementation and Working Principle

6. Conclusion

7. Outputs

1. Introduction :

Electronic commerce (e-commerce) has revolutionized modern retail, enabling consumers to discover, evaluate, and purchase thousands of products seamlessly across web platforms. Modern e-commerce platforms handle massive product catalogs requiring rapid search, multi-faceted filtering, real-time cart calculations, intelligent product recommendations, and optimal logistics management. Delivering responsive search interactions and sub-second catalog navigation without excessive computational overhead requires well-designed, highly optimized underlying data structures and algorithms (DSA).

KartShelf (E-Commerce Product Search & DSA Engine) is an end-to-end web-based e-commerce platform developed to demonstrate the practical implementation of core Data Structures and Algorithms. The application provides essential retail features including as-you-type search autocomplete, multi-criteria price and category filtering, interactive shopping cart management, multi-algorithm catalog sorting, "Customers Also Bought" recommendation graphs, and automated budget bundle generation. Every core subsystem is driven by custom, scratch-implemented data structures in pure Python, without relying on built-in data-structure libraries for the core DSA modules.

The project incorporates a comprehensive range of data structures, including arrays, static and dynamic stacks, circular queues, input- and output-restricted deques, header linked lists, circular linked lists, binary search trees (BST), threaded binary search trees, self-balancing AVL trees, multi-way B-Trees, B+ Trees with sequential leaf linkage, binary max-heaps, hash tables with multiple collision resolution mechanisms, prefix tries, disjoint sets (union-find), and graphs. Searching and sorting algorithms—including Quick Sort, Merge Sort, Heap Sort, Shell Sort, and Counting Sort—organize catalog items while providing real-time comparison metrics and sort stability analysis. Graph exploration techniques, including Breadth-First Search (BFS) and Depth-First Search (DFS), power product relationship discovery, while Minimum Spanning Tree (MST) algorithms (Prim’s and Kruskal’s) optimize delivery routes across fulfillment networks.

The application is built using a modern full-stack architecture comprising a Python Flask backend serving RESTful API endpoints and a responsive, neo-editorial web frontend constructed with semantic HTML5, modern CSS3, and vanilla JavaScript.

The primary objective of this project is to bridge theoretical computer science principles and practical software engineering through a realistic e-commerce simulation. It provides a structured understanding of algorithmic complexity, data organization trade-offs, and memory management, demonstrating how fundamental DSA concepts directly empower high-performance retail software systems.

2. Problem Statement and Objectives

2.1 Problem Statement

Managing a large-scale e-commerce product catalog involves handling thousands of stock-keeping units (SKUs), providing instant search autocomplete, enabling complex multi-attribute filtering (by category, price range, and brand), sorting large product collections, tracking cart totals in real time, and recommending companion products based on shopping patterns. Naive implementations relying on unindexed arrays or linear searches suffer from severe performance bottlenecks, leading to O(N) query latencies, sluggish response times, and unoptimized memory usage. A systematic, engineered application is required to organize product data efficiently, eliminate latency during high-frequency searches, maintain shopping cart integrity, and demonstrate how classical data structures—such as Tries, AVL Trees, B+ Trees, Heaps, and Graphs—solve practical computational challenges. Therefore, KartShelf (E-Commerce Product Search & DSA Engine) is developed to simulate commercial retail workflows while providing a transparent, rigorous demonstration of Data Structures and Algorithms (DSA).

2.2 Objectives

The main objectives of this project are:

- To develop a responsive, full-stack web application for e-commerce product discovery, search autocomplete, catalog filtering, and shopping cart management.
- To implement high-speed prefix search autocomplete using a custom Prefix Trie data structure with highlighted character matching.
- To implement balanced AVL Trees and B/B+ Trees for guaranteed O(log N) product ID lookups and fast sequential leaf price-range queries.
- To provide a multi-algorithm sorting engine featuring Quick Sort, Merge Sort, Heap Sort, Shell Sort, and Counting Sort, with live comparison counters and stability verification.
- To maintain active shopping cart state and compute grand totals in O(1) constant time using a custom Header Linked List.
- To manage search history and recently viewed products using Circular Queues and Restricted Deques.
- To model product recommendation networks and warehouse delivery routes using Graphs with BFS, DFS, and Minimum Spanning Tree (Prim's and Kruskal's) algorithms.
- To implement custom Hash Tables supporting multiple hash functions (Division, Mid-Square, Folding) and collision resolution strategies (Linear Probing, Quadratic Probing, Separate Chaining).
- To solve optimal budget bundle item selection by maximizing total rating within a price budget using Backtracking search with branch-and-bound pruning rules.
- To analyze theoretical time and space complexities of the implemented data structures and algorithms.

3. System Requirements and Technologies Used

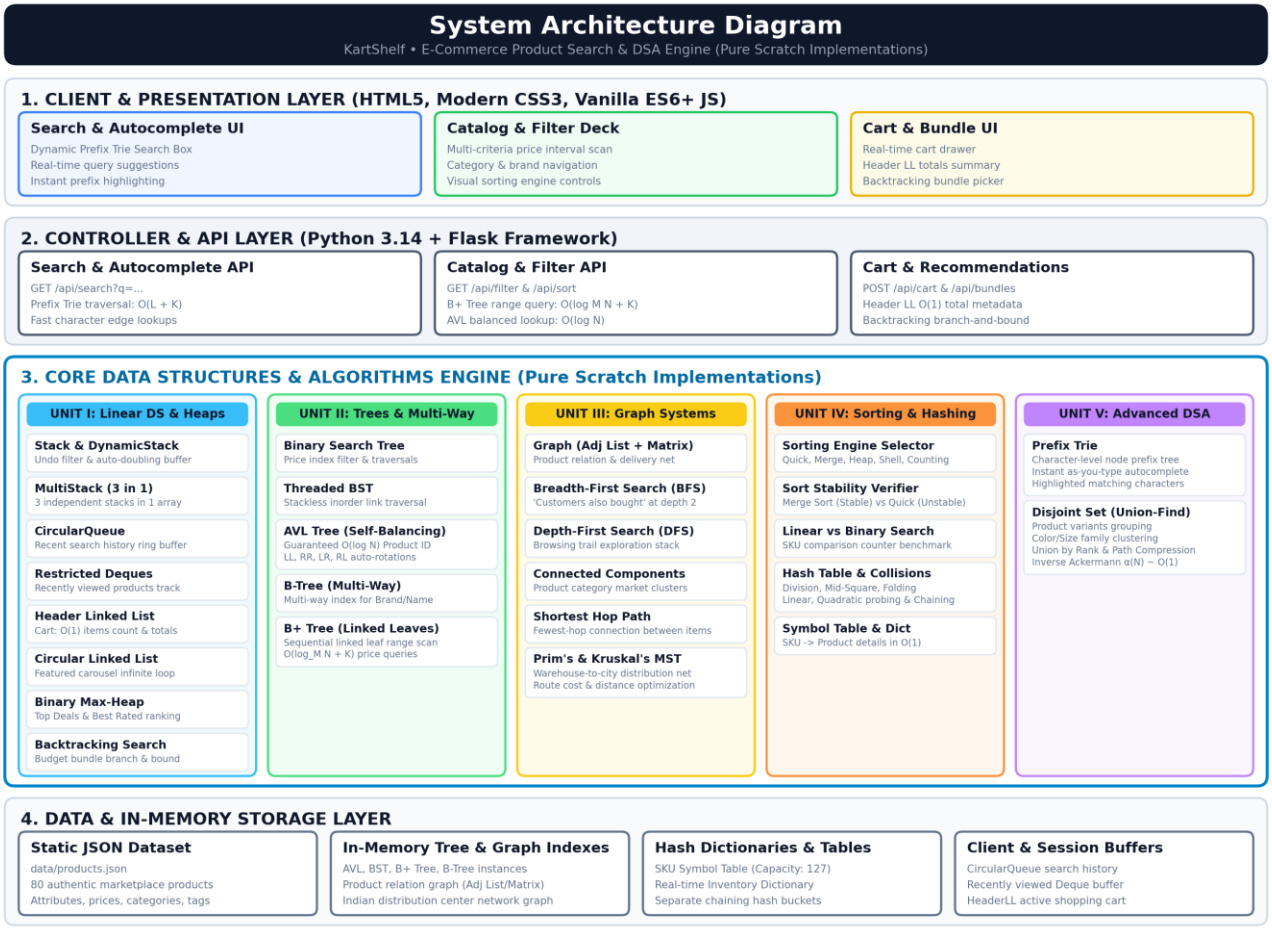
3.1 Hardware Requirements

A computer capable of running a modern web browser and a current Python 3.14+ execution environment is sufficient for local demonstration and evaluation. No dedicated GPU or specialized server hardware is required.

3.2 Software and Tools

- Frontend: Plain HTML5, Modern CSS3 with Neo-Editorial design tokens, and Vanilla JavaScript (ES6+).
- Backend: Python 3.14+ with lightweight Flask web framework serving templates and RESTful APIs.
- Core DSA Library: Custom pure-Python implementations of the core data structures and algorithms, built from scratch without built-in data-structure libraries.
- Data Persistence: JSON dataset (products.json), in-memory index structures, and browser session state.
- Testing: Automated Pytest suite (35 unit tests validating the core data structure operations).
- Version Control: Git and GitHub.

4. Architecture Diagram



5. Implementation and Working Principle

5.1 Implementation

The major modules include:

- **Prefix Search & Autocomplete:** Uses a Prefix Trie to deliver instant search suggestions with highlighted matching prefixes as the user types.
- **Catalog Indexing & Range Filtering:** Implements AVL Trees and B+ Trees for logarithmic product lookup and sequential linked leaf scans for price-range filtering.
- **Multi-Algorithm Sorting Engine:** Offers selectable sorting algorithms (Quick, Merge, Heap, Shell, Counting) with comparison counters and stability verification.
- **Cart & Infinite Carousel:** Utilizes a Header Linked List for O(1) cart metadata updates and a Circular Linked List for featured boutique product carousels.
- **Priority Ranking & Top Deals:** Employs a Binary Max-Heap to rank products by discount percentage and customer rating.
- **Recommendation & Distribution Graph:** Uses adjacency lists and matrices with BFS for product recommendations and Prim/Kruskal algorithms for warehouse routing.
- **Hash Table & Stock Registry:** Demonstrates Division, Mid-Square, and Folding hash functions with chaining and probing collision resolution for instant SKU lookups.
- **Budget Bundle Builder:** Solves optimal item selection under a price budget using backtracking search with branch-and-bound pruning rules.
- **Undo/Redo, History & Clustering:** Uses Stacks for filter undo/redo, Circular Queues and Restricted Deques for search and recently viewed history, BST and Threaded BST for price-ordered traversal, and Disjoint Sets (Union-Find) for grouping product variants.

5.2 Working Principle

Step 1: Accessing the Application: The user opens the web application in a browser at http://127.0.0.1:5000 to access the storefront.

Step 2: Searching Products via Trie Autocomplete: As the user enters query terms, the Prefix Trie traverses matching character nodes in O(L + K) time, returning autocompleted suggestions with prefix highlighting.

Step 3: Managing and Filtering Product Catalog: When filters are applied, the B+ Tree performs efficient range scans along linked leaf nodes, while the AVL Tree performs balanced logarithmic lookups by product ID.

Step 4: Sorting Products and Complexity Telemetry: The user chooses a sorting algorithm (Merge, Quick, Heap, Shell, Counting). The system sorts the catalog, tracks comparison counts, and demonstrates sort stability.

Step 5: Managing Cart and Navigation History: Adding products updates the Header Linked List in O(1) for grand totals. Stacks track filter history for undo/redo, while Circular Queues track recent searches.

Step 6: Exploring Graph Recommendations and Bundles: On product pages, Graph BFS identifies related items up to depth 2. In the cart, the Backtracking Bundle Builder recommends optimal items maximizing rating within a budget.

6. Conclusion

KartShelf (E-Commerce Product Search & DSA Engine) demonstrates the practical application of Data Structures and Algorithms (DSA) in a modern e-commerce context through a comprehensive web-based platform. The project provides a structured approach to managing product catalogs, executing high-speed autocomplete searches, filtering price intervals, ranking promotional deals, maintaining shopping carts, and navigating recommendation graphs.

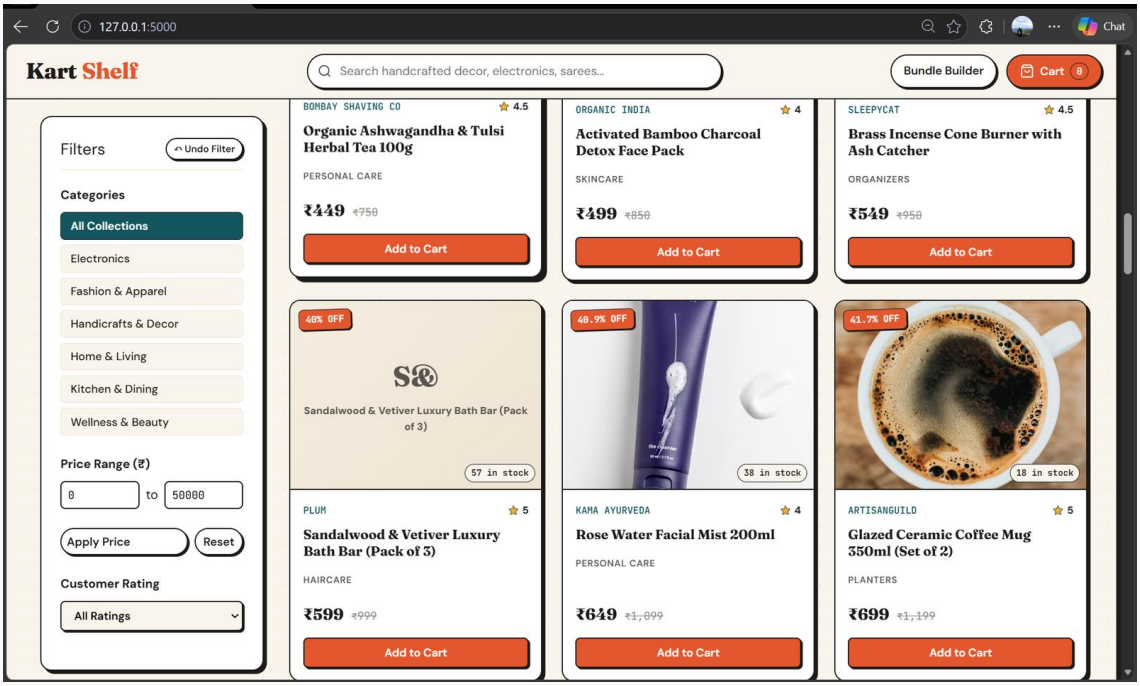
Various data structures, including arrays, static and dynamic stacks, circular queues, restricted deques, header linked lists, circular linked lists, binary search trees, threaded BSTs, AVL trees, B-Trees, B+ Trees, heaps, hash tables, prefix tries, disjoint sets, and graphs, illustrate how complex commercial data can be organized, indexed, and processed with optimal time and space efficiency. Searching and sorting techniques highlight performance trade-offs between O(N), O(log N), O(N + K) for Counting Sort, and average-case O(N log N) approaches, while graph algorithms (BFS, DFS, Prim's, and Kruskal's) provide powerful tools for relationship clustering and delivery logistics optimization. The project emphasizes the vital role of algorithmic complexity and clean software design in building responsive, scalable applications.

Developing this project provides extensive practical experience in full-stack Python web development, algorithmic implementation from first principles, data modeling, computational testing, and user-centric visualization. It bridges the gap between theoretical textbook DSA concepts and their real-world software applications.

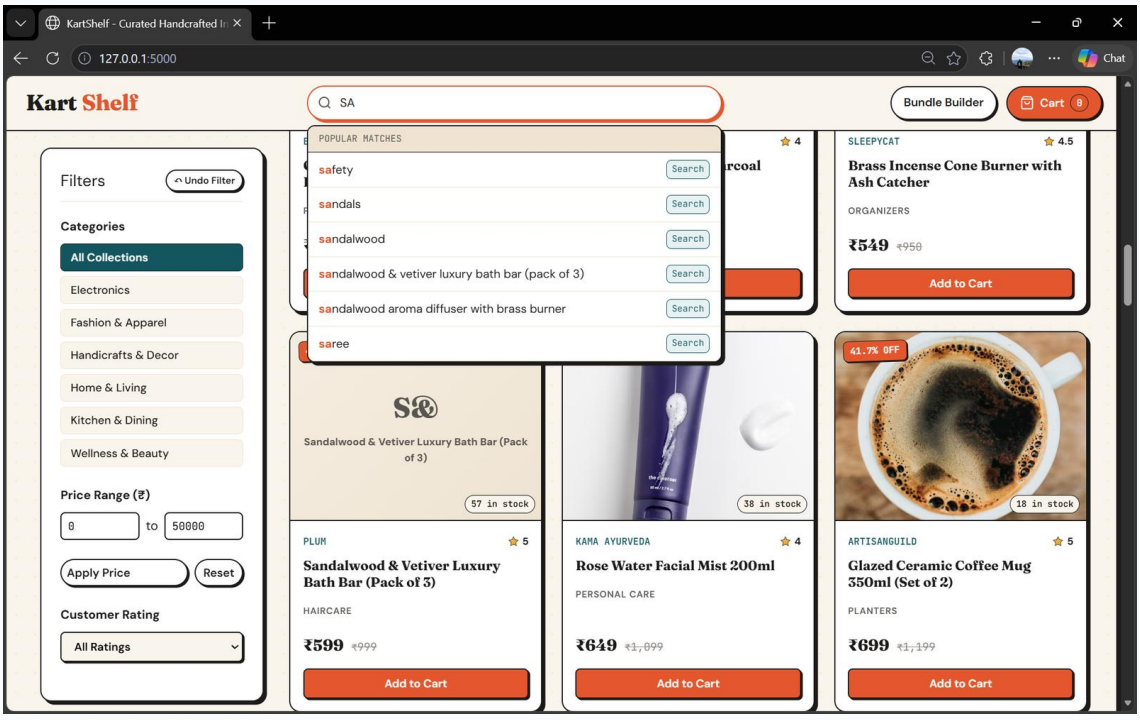
In conclusion, KartShelf serves as an effective educational platform for understanding how fundamental data structures and algorithms drive core features in commercial software systems. The project can be further extended in the future by incorporating persistent relational and NoSQL database layers, distributed cache hierarchies, machine-learning-based recommendation scoring, and multi-tenant authentication. The current system provides a complete, self-contained educational simulation designed for rigorous academic evaluation and learning.

7. Outputs

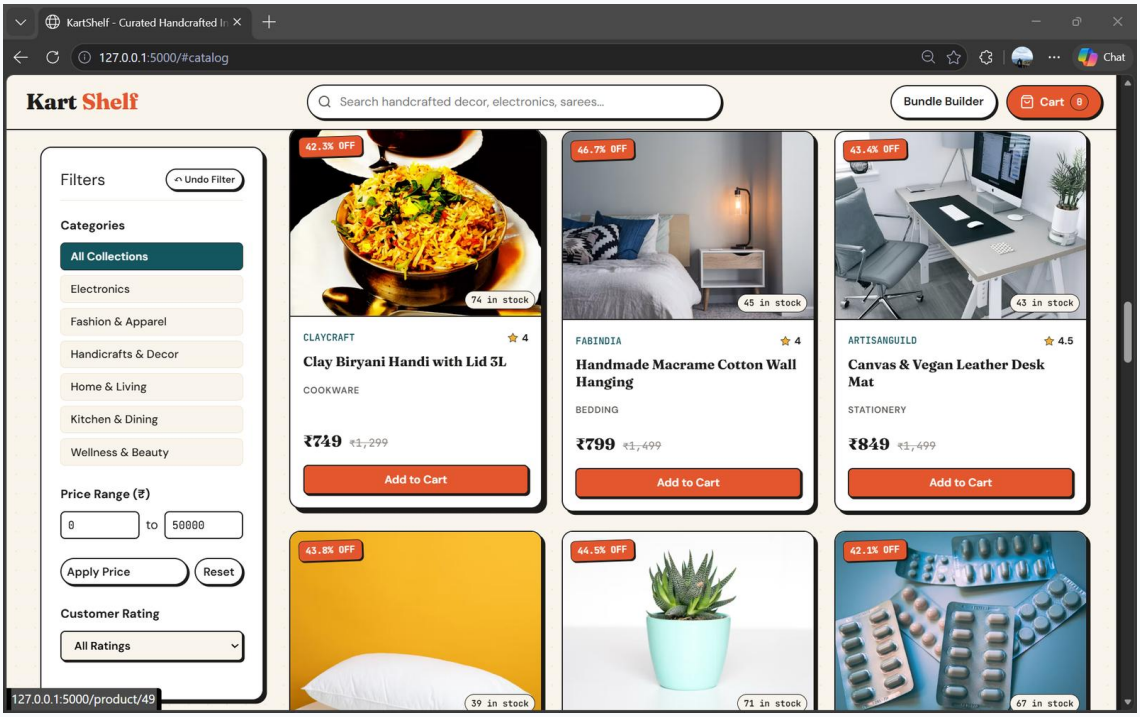
Output 1: Storefront Homepage & Product Catalog Listing



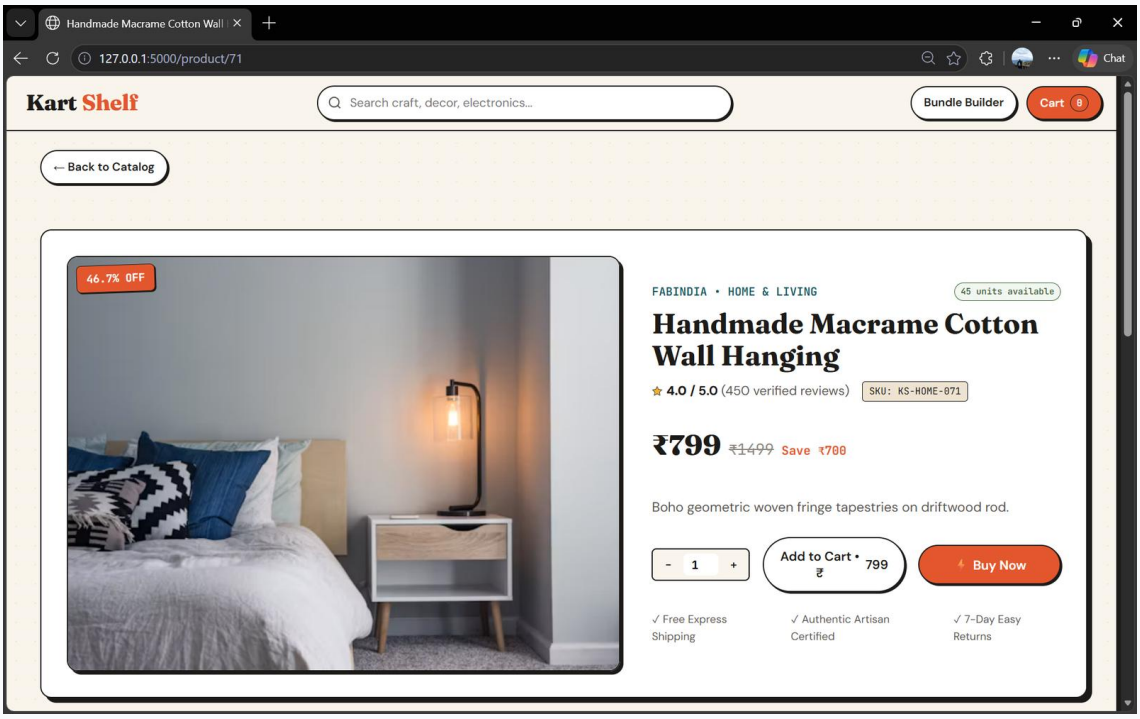
Output 2: As-You-Type Prefix Trie Search Autocomplete



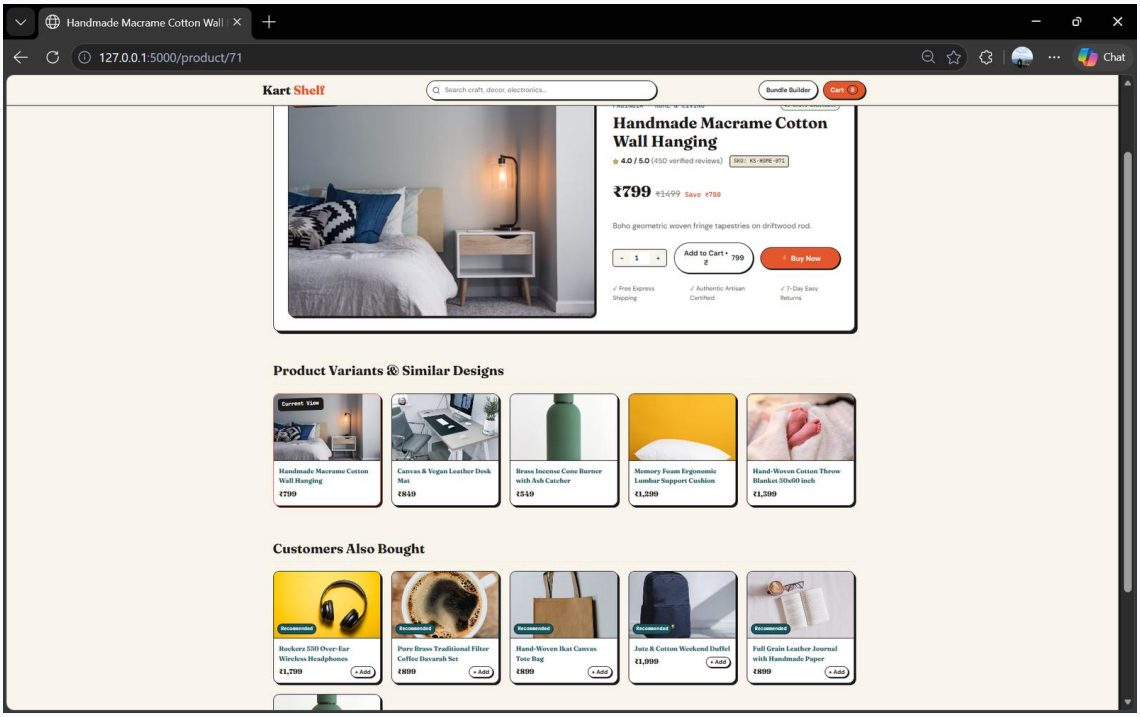
Output 3: Product Catalog with Category & Price Range Filters (Undo Filter Stack)



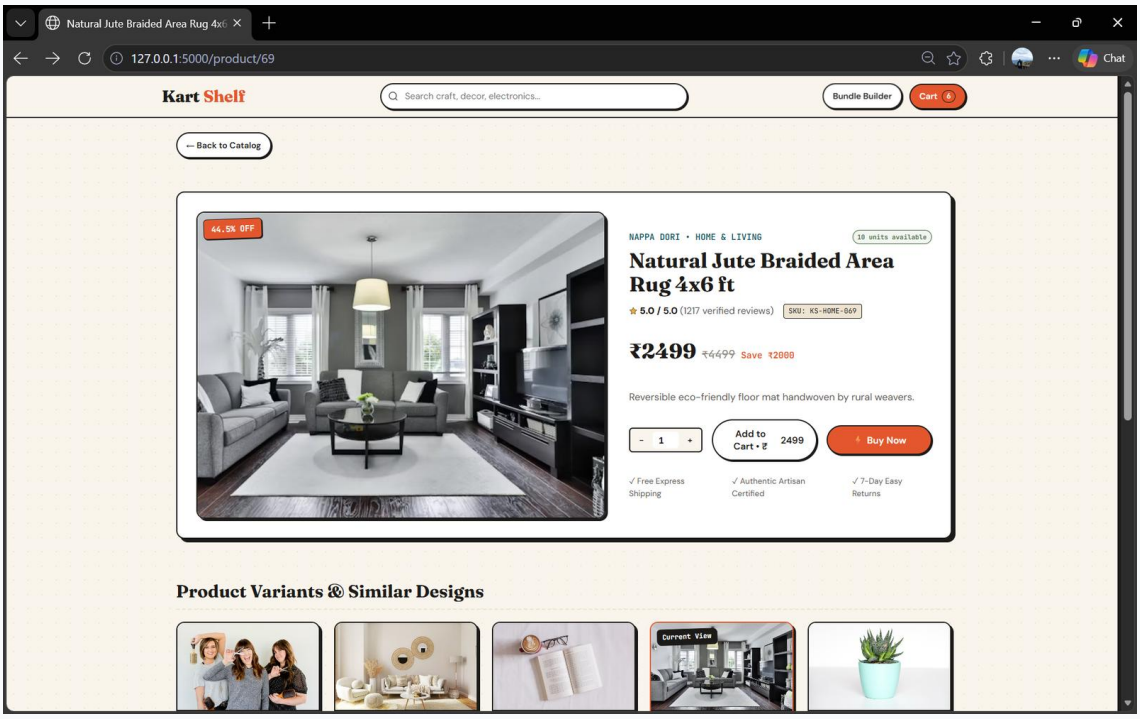
Output 4: Product Detail View



Output 5: Product Variants & Graph BFS Recommendations ("Customers Also Bought")



Output 6: Product Detail View with Product Variants



Output 7: Interactive Shopping Cart (Header Linked List) & Budget Combo Builder

